

VecFormer: Towards Efficient and Generalizable Graph Transformer with Graph Token Attention

Jingbo Zhou[†]
Zhejiang University
Hangzhou, Zhejiang, China
Westlake University
Hangzhou, Zhejiang, China
zhoujingbo@westlake.edu.cn

Jun Xia[†]
Hong Kong University of Science and
Technology (Guangzhou)
Guangzhou, Guangdong, China
junxia@hkust-gz.edu.cn

Siyuan Li[†]
Westlake University
Hangzhou, Zhejiang, China
lisiyuan@westlake.edu.cn

Yunfan Liu
Westlake University
Hangzhou, Zhejiang, China
liuyunfan@westlake.edu.cn

Wenjun Wang
Tongji University
Shanghai, China
2251275@tongji.edu.cn

Yufei Huang
Westlake University
Hangzhou, Zhejiang, China
huangyufei@westlake.edu.cn

Changxi Chi
Westlake University
Hangzhou, Zhejiang, China
chichangxi@westlake.edu.cn

Mutian Hong
ShanghaiTech University
Shanghai, China
hongmt2022@shanghaitech.edu.cn

Zhuoli Ouyang
Southern University of Science and
Technology
Shenzhen, Guangdong, China
ouyangzl2023@mail.sustech.edu.cn

Shu Wang
Jilin University
Changchun, Jilin, China
168778017@qq.com

Zhongqi Wang
Jilin University
Changchun, Jilin, China
wangzq2024@mails.jlu.edu.cn

Xingyu Wu
Zhejiang University
Hangzhou, Zhejiang, China
wuxingyu@zju.edu.cn

Chang Yu[†]
Westlake University
Hangzhou, Zhejiang, China
yuchang@westlake.edu.cn

Stan Z. Li^{*}
Westlake University
Hangzhou, Zhejiang, China
stan.zq.li@westlake.edu.cn

Abstract

Graph Transformer has demonstrated impressive capabilities in the field of graph representation learning. However, existing approaches face two critical challenges: (1) most models suffer from exponentially increasing computational complexity, making it difficult to scale to large graphs; (2) attention mechanisms based on node-level operations limit the flexibility of the model and result in poor generalization performance in out-of-distribution (OOD) scenarios. To address these issues, we propose **VecFormer** (the **Vector Quantized Graph Transformer**), an efficient and highly generalizable model for node classification, particularly under OOD settings. VecFormer adopts a two-stage training paradigm. In the first stage, two codebooks are used to reconstruct the node features and the graph structure, aiming to learn the rich semantic Graph

Codes. In the second stage, attention mechanisms are performed at the Graph Token level based on the transformed cross codebook, reducing computational complexity while enhancing the model's generalization capability. Extensive experiments on datasets of various sizes demonstrate that VecFormer outperforms the existing Graph Transformer in both performance and speed.

CCS Concepts

• **Computing methodologies** → **Machine learning**; • **Mathematics of computing** → *Graph algorithms*.

Keywords

Graph Representation Learning, Graph Transformer, Node Classification, Graph OOD, Vector Quantization

ACM Reference Format:

Jingbo Zhou[†], Jun Xia[†], Siyuan Li[†], Yunfan Liu, Wenjun Wang, Yufei Huang, Changxi Chi, Mutian Hong, Zhuoli Ouyang, Shu Wang, Zhongqi Wang, Xingyu Wu, Chang Yu[†], and Stan Z. Li. 2026. VecFormer: Towards Efficient and Generalizable Graph Transformer with Graph Token Attention. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792453>

^{*}Stan Z. Li is the corresponding author.

[†]These authors contributed equally to this work.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792453>

1 Introduction

Graph data is prevalent in various domains [3–5, 7, 18, 32, 47, 53] such as social networks, bioinformatics, multimodality and transportation systems. Graph Neural Networks (GNNs) [19, 36] learn node representation from graph structure through the message-passing mechanism, achieving significant success in many graph machine learning tasks. However, GNNs still face limitations in capturing global information and long-range dependencies [10, 44, 57]. To address these challenges, Graph Transformers [46, 52, 56] have emerged, drawing inspiration from the success of Transformer in natural language processing. Graph Transformers can effectively extract global features by leveraging the self-attention mechanism to model complex relationships between nodes.

Graph Transformers have shown potential in handling graph-structured data, but they face significant limitations regarding efficiency and generalization in the node classification task. **First, they typically rely on the self-attention mechanism, which results in a computational complexity of $O(N^2)$** , where N is the number of nodes in the graph. This presents challenges in terms of memory and computational resources when processing large-scale graphs. To address this computational bottleneck, previous works fall into two categories: (1) employing linear attention to reduce the complexity of computing attention through matrix multiplication [42, 43], and (2) introducing global or partitioned token nodes, enabling each node to interact with these token nodes to obtain global information [2, 12, 59]. However, each of the above methods has its drawbacks. Linear attention-designed mapping functions often struggle to achieve the expressiveness of the self-attention mechanism in most datasets. Introducing global or local token nodes explicitly retains global information, but it becomes challenging to learn finer-grained relationships between nodes. **Secondly, current methods overly rely on node-level attention learning, which affects their generalizability in out-of-distribution (OOD) scenarios.** Existing approaches perform attention at the node-level, struggling to capture useful patterns due to distributional differences between the training and test sets in the OOD scenarios, which impacts the model’s generalization ability. Hence, a natural question emerges: *Can we design an efficient and generalizable Graph Transformer?*

To answer this question, instead of designing a more complex Graph Transformer that combines multiple levels of attention mechanisms, we propose breaking away from the current node-level attention framework and modeling attention at the graph token level, which represents the distribution of graph data. More specifically, in the VecFormer’s first training stage, we introduce a variant of VQVAE[34], SoftVQ[14], to quantize the GNN output to learn the Graph Codes with rich semantic information by reconstructing both the node features and the graph structure. In the second stage, we combine the codes from the transformed structure codebook and feature codebook pairwise to obtain a cross-codebook that constructs Graph Tokens. We then perform cross-attention between the quantized node representations and all Graph Tokens in the cross-codebook. The output is combined with the quantized node representations to form the final output of the model. By introducing soft vector quantization into the Graph Transformer, we effectively compress the information distribution in the graph,

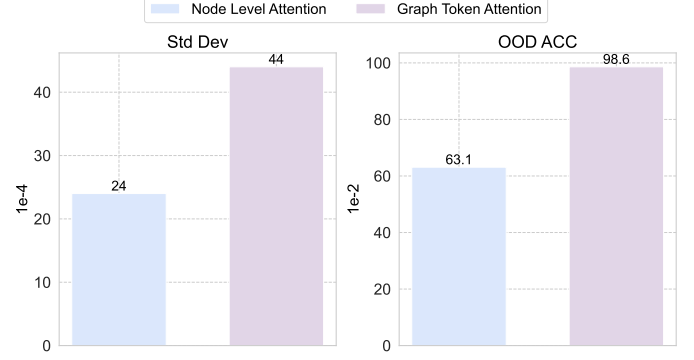


Figure 1: Standard deviation of attention weights and performance evaluation of Node Level Attention and Graph Token Attention in OOD setting.

reducing the computational complexity. Moreover, we theoretically and experimentally demonstrate the superiority of the graph token attention mechanism, enabling VecFormer to capture beneficial patterns and enhancing the generalization ability in OOD scenarios. We highlight our contributions as follows.

- **Significance:** We emphasize the importance of attention on general Graph Token that represents the data distribution, to move away from node-level attention, which limits the efficiency and generalization of Graph Transformers.
- **Algorithm:** We developed a novel algorithm, VecFormer, which uses soft vector quantization to reduce the attention computation complexity on graphs while improving the generalization ability of Graph Transformers in OOD scenarios.
- **Experiments:** We experimentally show that the proposed method outperforms other Graph Transformers in terms of node classification task, OOD scenarios, and efficiency.

2 Related Work

2.1 Graph Transformer

Graph Transformers (GTs) have emerged as a powerful approach for graph representation learning, leveraging attention mechanisms to capture complex relationships within graph-structured data. Various methodologies have been proposed to enhance their efficiency and scalability. NodeFormer [42] introduces a kernelized Gumbel-Softmax operator to facilitate efficient all-pair message passing with linear complexity, making it suitable for large-scale node classification tasks. Similarly, SGFormer [43] simplifies the architecture by employing a single-layer global attention mechanism, achieving significant inference acceleration without complex preprocessing. GOAT [20] enhances the adaptability of node relationships by allowing each node to attend to all others in the graph, effectively addressing homophily and heterophily in node classification. GraphGPS [26] presents a modular framework that integrates local message passing with global attention, maintaining linear complexity while achieving state-of-the-art performance across various tasks. Exphormer [31] leverages expander graphs and sparse attention to enhance scalability, achieving linear time

complexity while maintaining competitive accuracy. Polynormer [8] further advances the field by employing a local-to-global attention mechanism that learns high-degree polynomial functions from node features efficiently. Lastly, NAGphormer [2] innovatively treats nodes as sequences of tokens through its Hop2Token module, enabling scalable multi-hop neighborhood aggregation. In contrast to these node-level attention methods, VecFormer applies graph token attention using vector quantization techniques.

2.2 OOD Generalization on Graph

Out-of-distribution (OOD) generalization in Graph Neural Networks (GNNs) is a critical area of research, as GNNs often struggle when the test data distribution differs from the training data [49–51]. Various approaches have been proposed to tackle this challenge. Invariant Risk Minimization (IRM) [1] aims to capture invariant correlations across multiple training distributions, providing a framework for learning robust representations. Deep Coral [33] extends the original Coral method to learn nonlinear transformations that align layer activation correlations, enhancing model adaptability to distribution shifts. Domain-Adversarial Neural Networks (DANN) [13] introduce domain-adapted representations, bridging the gap between similar yet distinct training and testing distributions. Group Distributionally Robust Optimization (GroupDRO) [28] emphasizes regularization for worst-group generalization, achieving improved accuracy by combining group DRO with increased regularization. Mixup [55] enhances the training dataset by interpolating between observed samples, thereby promoting robustness against distributional discrepancies. The SR-GNN [58] model addresses these discrepancies specifically in graph contexts by adjusting GNN architectures to account for shifts between labeled nodes in the training set and the broader dataset. EERM [41] employs adversarial training based on invariance principles to facilitate environmental exploration. Recent advancements like CaNet [40] focus on preserving causal invariance among nodes to bolster OOD generalization capabilities. VecFormer narrows the distributional discrepancy between the training and test sets through the SoftVQ technique, thereby enhancing the generalization ability of the attention mechanism.

2.3 Vector Quantization in Graph Neural Networks

Recent advances in vector quantization (VQ) techniques have significantly advanced graph representation learning across biological domains. The foundational work of VQ-GNN[9] introduced a learnable codebook mechanism to map continuous node embeddings into discrete semantic spaces, employing hierarchical quantization and attention-based code assignment to balance structural preservation with computational efficiency in large-scale graph processing. Mole-BERT[45] revolutionized molecular graph pre-training through a type-constrained VQ-VAE architecture, where specialized codebook partitions for different atom types (e.g., dedicated ranges for carbon/nitrogen/oxygen) prevent cross-type interference while enabling context-aware discretization of atomic environments. This framework synergized masked atom modeling with triplet contrastive learning to capture both local reconstructive and global relational patterns. The Prompt-DDG[39] established a hierarchical prompt codebook system that decomposes

3D microenvironments into residue-type, angular statistics, and local conformation subspaces, enabling efficient mutation effect prediction through masked microenvironment modeling. Concurrently, the MAPE-PPI[38] framework achieved state-of-the-art performance by integrating VQ-derived discrete prompts that encode microenvironmental variations with minimal parameter overhead (<5%). VQGraph[48] propose a dual-channel codebook architecture to bridge GNNs and MLPs through discrete-continuous hybrid representations. This design enables hard index selection from one codebook channel while performing soft embedding retrieval from another, effectively aligning GNN’s structural awareness with MLP’s inference speed. In this work, we focus on enhancing the GT’s computational efficiency and the generalization ability in OOD scenarios through vector quantization techniques.

3 Preliminary

A connected undirected graph, denoted by $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, consists of the set of N nodes $\mathcal{V} = \{v_1, v_2, \dots, v_N\}$ and the set of edges $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$. The node features are represented in the matrix $X \in \mathbb{R}^{N \times d}$, where x_i denotes the feature of node v_i and d represents the dimension of the feature. $A \in \{0, 1\}^{N \times N}$ denotes the adjacency matrix, where $A_{ij} = 1$ signifies the presence of an edge between nodes v_i and v_j in the graph. The adjacency matrix with self-loop is represented as $\hat{A} = A + I$. Additionally, we use H to denote the output of the GNN layer, representing the new representation matrix of node features.

3.1 Graph Transformer

Graph Transformers leverage a global attention mechanism to enable each node to interact with every other node in the graph. The node features H are updated as follows:

$$Q = HW_Q, \quad K = HW_K, \quad V = HW_V, \quad (1)$$

$$\text{Attention}(H) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V, \quad (2)$$

where N is the number of nodes, $W_Q, W_K, W_V \in \mathbb{R}^{d \times d}$ are learnable projection matrices. The attention scores are calculated using the scaled dot-product of Q and K , normalized with the softmax function. These scores are then used to aggregate information from all nodes into V , enabling effective global information propagation.

3.2 Vector Quantization

Vanilla VQ The VQ-VAE model, introduced by van den Oord et al. [34], is designed to model continuous data distributions, such as images, audio, and video, by encoding them into discrete latent variables. The encoder maps the input x to a continuous latent vector $E(x)$, which is then quantized to the nearest codebook vector e_k based on a minimum distance criterion:

$$\text{Quantize}(E(x)) = e_k, \quad k = \arg \min_j \|E(x) - e_j\|. \quad (3)$$

The model is trained using the reconstruction loss, which measures the difference between the input x and the decoder output $D(e)$, alongside two additional terms: the codebook loss, which ensures that the selected codebook vector is close to the encoder output, and the commitment loss, which stabilizes the encoder by encouraging

$E(x)$ to stay close to the chosen codebook vector. The total loss function is expressed as:

$$\mathcal{L}(x, D(e)) = \|x - D(e)\|_2^2 + \|\text{sg}[E(x)] - e\|_2^2 + \eta \|\text{sg}[e] - E(x)\|_2^2, \quad (4)$$

where sg denotes the stop-gradient operation, and η is a hyper-parameter that controls the relative learning speed.

4 VecFormer

In this section, we provide a detailed introduction to VecFormer. The core idea of VecFormer is to discretely compress the representational space of feature and structural distributions on graph, learn semantically rich Graph Codes, and apply them in a Transformer with a graph token attention mechanism. This approach enhances the model's generalization ability while reducing computational efficiency. As shown in Figure 2, the training of this model consists of two stages: **Graph Codebook Training** and **Graph Transformer Finetuning**. The basic notations related to VecFormer and their meanings are listed in Table 1.

Table 1: Basic notation and their meanings.

Notation	Meaning
$\mathcal{G}$	Attribute Graph
N	Number of nodes
A	Adjacency matrix
T	Temperature parameter
$X \in \mathbb{R}^{n \times d}$	Feature matrix
$H \in \mathbb{R}^{n \times d'}$	GNN's output matrix
$F_C = \{f_{c_z}\}_{z=1}^m$	Feature Codebook
$S_C = \{s_{c_z}\}_{z=1}^n$	Structure Codebook
G	The nodes' corresponding Graph Token
F_T, S_T, G_T	Feature, Structure and Graph Token List

4.1 Graph Codebook Training

In the Graph Codebook Training stage, we employ two codebooks: a Feature Codebook and a Structure Codebook. Using the SoftVQ approach, we define self-supervised tasks including feature reconstruction, structure reconstruction, and joint reconstruction of graph features. These tasks aim to learn semantically rich Graph Codes, consisting of Feature Codes and Structure Codes. For each node, the corresponding Feature Token and Structure Token are obtained through a linear combination of codes from the Feature Codebook and Structure Codebook using the SoftVQ method. The Graph Token for the node is generated by fusing the Feature Token and Structure Token through a Token Fusion module. This process effectively quantizes the node and derives its corresponding Graph Token, providing the foundation for the Graph Token-level attention mechanism in the Graph Transformer Finetuning stage.

Soft VQ Vanilla VQ is prone to codebook collapse during training, where the model quantizes all samples to a few codebook vectors, leading to training failure. To address this, we introduce the SoftVQ to improve vector quantization by introducing a differentiable attention mechanism. Instead of selecting a single closest codebook vector, SoftVQ computes a set of attention weights a_{ij}

that quantify the contribution of each codebook vector e_j to the input feature h_i . The final output $\hat{h}_i$ is computed as a weighted sum of the codebook vectors:

$$a_{ij} = \frac{\exp(h_i^T e_j / T)}{\sum_{j=1}^m \exp(h_i^T e_j / T)}, \quad \hat{h}_i = \sum_{j=1}^m a_{ij} e_j, \quad (5)$$

where T is a temperature parameter that controls the sharpness of the attention distribution, with higher values leading to more even contributions from the codebook vectors. This approach allows for a smoother, more flexible representation compared to hard quantization.

Using Feature and Structure Codebooks to Learn Graph Data Distribution VQ-VAE has been widely applied in computer vision (CV). Unlike images, which are Euclidean data, graphs are non-Euclidean data with two dimensions: features and structure. In VecFormer, we employ a Feature Codebook and a Structure Codebook to learn the feature and structural distributions of graph data. Given a graph $\mathcal{G}$ with nodes $\mathcal{V} = \{v_1, v_2, \dots, v_N\}$, each node has corresponding features $\{x_1, x_2, \dots, x_N\}$. Let the **Feature Codebook** be $\{f_{c_z}\}_{z=1}^m$ and the **Structure Codebook** be $\{s_{c_z}\}_{z=1}^n$. For each node, the corresponding Feature Token and Structure Token are obtained through an attention-weighted linear combination of codes from the Feature Codebook and Structure Codebook. The quantization process can be formally expressed as follows:

$$f_i = \text{SoftVQ}(h_i, \{f_{c_z}\}_{z=1}^m), \quad s_i = \text{SoftVQ}(h_i, \{s_{c_z}\}_{z=1}^n), \quad (6)$$

where h_i is the node representation output by the GNN, and f_i and s_i are the Feature Token and Structure Token obtained through quantization. In VecFormer, the GNN encoder can be any Message Passing Neural Network (MPNN) framework. **It is important to note that graph features and structures are not entirely orthogonal.** For instance, nodes of different class often have distinct neighborhood structures, necessitating the joint modeling of feature and structure distributions. Furthermore, to obtain an overall quantized representation of a node, rather than independent Feature Token and Structure Token, we employ a **Token Fusion** module to combine the two tokens into a Graph Token. The fusion process is as follows:

$$g_i = \alpha_1 \cdot f_i + \alpha_2 \cdot s_i, \quad (\alpha_1, \alpha_2) = \varphi_w(f_i, s_i), \quad (7)$$

where φ_w is a linear transformation, g_i is the Graph Token obtained by fusing the Feature Token f_i and the Structure Token s_i . We optimize the codebooks using these three types of tokens.

Graph Codebook Optimization To obtain semantically rich Graph Codes, we optimize the Feature Codebook and Structure Codebook through self-supervised tasks involving the reconstruction of node features, graph structure, and graph features. The loss for optimizing the Graph Codes in codebooks is as follows:

$$\begin{aligned} \mathcal{L}_{rec} = & \underbrace{\frac{1}{n} \sum_{i=1}^n \left(1 - \frac{x_i^T \hat{x}_i}{\|x_i\| \cdot \|\hat{x}_i\|} \right)^{Y_f}}_{\text{Feature Reconstruction}} + \underbrace{\|A - \sigma(\hat{Y} \cdot \hat{Y}^T)\|_2^2}_{\text{Structure Reconstruction}} \\ & + \underbrace{\frac{1}{n} \sum_{i=1}^n \left(1 - \frac{h_i^T \hat{h}_i}{\|h_i\| \cdot \|\hat{h}_i\|} \right)^{Y_g}}_{\text{Graph Reconstruction}}, \end{aligned} \quad (8)$$

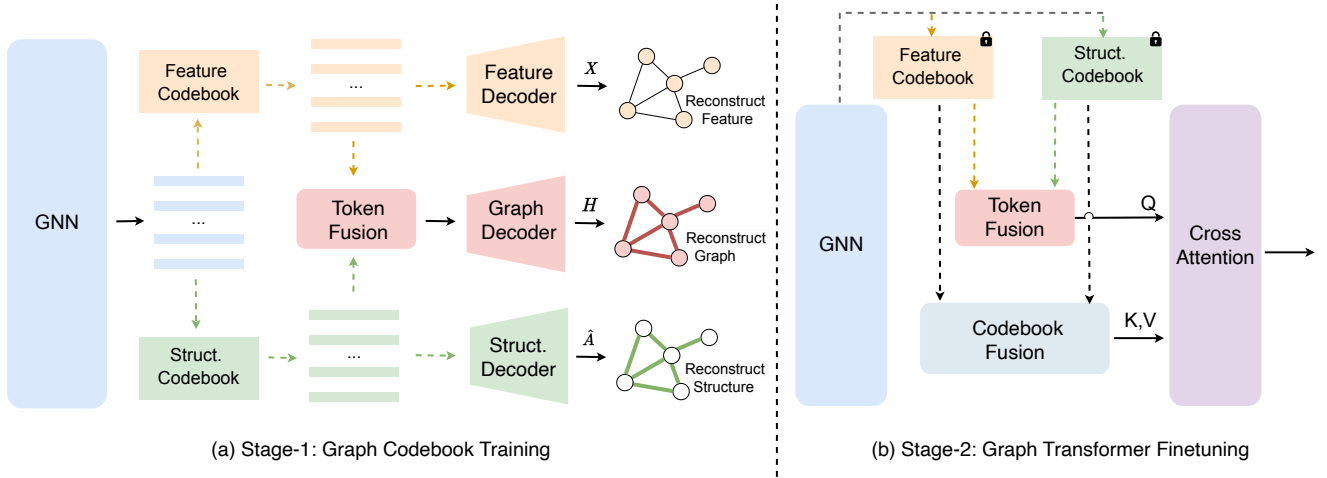


Figure 2: Illustration of the VecFormer's architecture and the training process.

where $\hat{x}_i$ represents the node features reconstructed from the Feature Token through the **Feature Decoder**, while $\hat{Y}$ is the matrix obtained from the Structure Token through the **Structure Decoder**, which is used to reconstruct the adjacency matrix A . $\hat{h}_i$ denotes the GNN output reconstructed from the Graph Token via the **Graph Decoder**, and σ refers to the Sigmoid function. All of the above decoders are implemented using the MLP. Since MSE is sensitive to vector norms and dimensions, we use the scaled cosine error adopted by GraphMAE [16] for feature and graph reconstruction. γ_f and γ_g are scaling factors. Through the graph feature reconstruction task, we acquire semantically rich feature and structure codebooks that enhance Graph Transformer Finetuning.

4.2 Graph Transformer Finetuning

In the Graph Transformer Finetuning phase, we utilize the pre-trained codebooks to construct a context-aware Graph Token List, which functions as both the key and value for the cross-attention mechanism. This enables the nodes' graph tokens to capture global graph information during attention interactions with the Graph Token List.

Constructing Context-aware Graph Token List Given the Feature Codebook $F_C = \{f_{c_z}\}_{z=1}^m$ and the Structure Codebook $S_C = \{s_{c_z}\}_{z=1}^n$ obtained in the Graph Codebook Training phase, the process for constructing the Graph Token List via **Codebook Fusion** module is as follows:

$$F_T = (F_C^T W_F)^T, \quad S_T = (S_C^T W_S)^T, \quad (9)$$

$$G_T = \text{TokenFusion}(F_T, S_T), \quad (10)$$

where F_T and S_T are the Feature and Structure Token List, $W_F \in \mathbb{R}^{m \times N_f}$ and $W_S \in \mathbb{R}^{n \times N_s}$ are linear projections. Since the Feature and Structure Token are linear combinations of the codes from codebooks that compress the graph distribution, we set learnable parameters W_F and W_S to transform the codebooks into a context-aware Token List. N_f and N_s denote the number of Feature Tokens and Structure Tokens. Then, we fuse the tokens pairwise from the Feature Token and Structure Token List

using the **Token Fusion** module, resulting in the context-aware Graph Token List with the size of $N_f N_s$.

Cross-Attention The nodes' corresponding Graph Token matrix G serves as the query for the attention mechanism. The Graph Token List G_T , constructed through the codebooks, functions as both the key and the value. The final output of the model is computed as follows:

$$Q = G W_Q, \quad K = G_T W_K, \quad V = G_T W_V, \quad (11)$$

$$Z = \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right) V + G, \quad (12)$$

where Z represents the model's output, $W_Q, W_K, W_V \in \mathbb{R}^{d \times d}$ are the learnable weight matrices. By applying attention between the Graph Token for node and the context-aware Graph Token List for the graph, the node's representation incorporates global information. The residual connection ensures that the output retains local information. The model is trained using a classification loss.

4.3 Model Analysis

In this section, we introduce how VecFormer reduces computational complexity and enhances the generalization ability of the attention mechanism by employing Graph Token Attention, as opposed to the node-level attention mechanism.

Model Computational Efficiency The computational complexity of node-level attention is $O(N^2)$. In VecFormer, we reduce the computational complexity to $O(NN_f N_s)$ by constructing a context-aware Graph Token List, which decreases the token count to $N_f N_s$, where N_f and N_s are typically small. This $N_f N_s$ is significantly smaller than N , which can reach millions of nodes, and remains unchanged as the graph size increases, achieving a linear computational complexity of $O(N)$. It is worth noting that both VecFormer training stages have linear complexity, and since the model converges quickly in the first stage, the overall training time is approximately equal to the training time of the second stage. In the meantime, unlike other linear complexity GTs, VecFormer reduces

computational complexity by minimizing the number of tokens requiring attention, which is orthogonal to methods that modify the attention mechanism. Furthermore, VecFormer’s attention architecture is simple, does not rely on complex kernel functions, is compatible with widely used attention mechanisms, and is more easily accelerated by hardware.

Model Generalization Ability In the Graph OOD scenarios, when performing self-attention mechanisms, q and k often come from distributions with significant differences. Due to differences in node distributions in the Graph OOD scenario and the fact that the OOD set is not included in the training set, resulting in the low similarity between q and k , the value of $q \cdot k$ is typically close to or equal to 0. When the lower bound of $q \cdot k$ is 0, the attention coefficients are computed as follows:

$$\text{Softmax}(qK) \sim U, \quad (13)$$

where U is uniform distribution. The importance of V remains constant across all values of Q , making it difficult for the attention mechanism to learn meaningful patterns. In VecFormer, since the Graph Token is a linear combination of the pretrained Graph Codes, we can express q and k as follows:

$$q = \sum_{i=1}^{m+n} \alpha_i c_i, \quad k = \sum_{i=1}^{m+n} \beta_i c_i, \quad (14)$$

where α, β denote the weight coefficients, and c represents the Feature and Structure Code. When the code is orthogonal to any other code, $q \cdot k$ can be computed as

$$q \cdot k \geq \sum_{i=1}^{m+n} \alpha_i \beta_i \|c_i\|^2 > 0. \quad (15)$$

The lower bound of $q \cdot k$ is no longer zero. VecFormer is capable of learning diverse attention patterns by adjusting α and β , thereby enhancing the model’s generalization ability. To further validate our point, we compare the standard deviation of learned attention weights and the performance of VecFormer and the vanilla Transformer on OOD datasets. As shown in Figure 1, in the OOD scenario, VecFormer exhibits a higher standard deviation in attention weights, suggesting more diverse attention patterns. Furthermore, VecFormer consistently achieves superior performance compared to the vanilla Transformer on OOD test sets. These results align well with our underlying explanation of the method’s effectiveness, providing empirical evidence that supports the validity of our design rationale.

5 Experiments

In this section, we comprehensively evaluate our proposed VecFormer by addressing the following research questions (RQs):

- **RQ1: Superiority.** Does VecFormer outperform other Graph Transformers on node classification tasks?
- **RQ2: Generalization.** Does our proposed method achieve greater improvements in graph out-of-distribution (OOD) scenarios?
- **RQ3: Scalability.** How does VecFormer perform in large graphs in terms of effectiveness and efficiency?
- **RQ4: Effectiveness.** Is the application of vector quantization techniques effective in Graph Transformers?

5.1 Node Classification Task (RQ1)

Setup. We evaluated VecFormer on node classification task using widely-used graph datasets. These include six homophilic graphs [29, 30]: CoraFull, Pubmed, Computer, Photo, CS, and Physics, as well as two heterophilic graphs [24], Chameleon and Squirrel, where redundant nodes were removed, as proposed in the benchmark [25]. Following the settings of prior works, we split the datasets into training, validation, and test sets in the ratio of 60%, 20%, and 20%, respectively. The mean and standard deviation of Accuracy were reported over ten runs. Detailed dataset information can be found in Appendix B.

Baselines. We compared VecFormer with three categories of GNN methods: standard GNNs, including GCN [19], GAT [36], and GraphSAGE [15]; recent Graph Transformer models, such as GOAT [21], GraphGPS [26], NodeFormer [42], SGFormer [43], Exphormer [31], NAGphormer [2], and Polynormer [8]; and vector quantization-based graph distillation methods, exemplified by VQGraph [48]. For VecFormer, we use GAT as the GNN encoder.

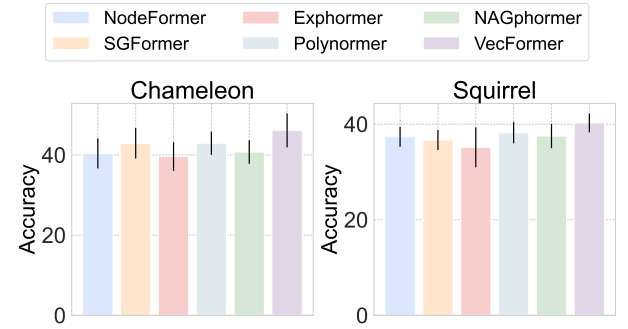


Figure 3: Experimental results(%) for node classification task on heterophily graph datasets Chameleon and Squirrel.

Results. The node classification results on homophilic graphs are presented in Table 2, while results on heterophilic graphs are shown in Figure 3. VecFormer outperforms standard GNNs, Graph Transformer models, and the graph distillation method VQGraph on the node classification task. Notably, VecFormer achieved greater improvements on heterophilic graphs compared to homophilic graphs. Compared to the homophilic graphs, which neighboring nodes often share the same label as the central node, the test sets of heterophilic graphs typically exhibit neighborhood structures different from those in the training set. Consequently, Graph Transformers with strong generalization and representational capabilities tend to perform better. VecFormer, along with Polynormer, which possesses polynomial-expressive, demonstrated outstanding performance on the two heterophilic graph datasets.

5.2 OOD Generalization Scenario (RQ2)

Setup. We select two types of OOD datasets to evaluate the effectiveness of VecFormer in OOD scenarios. Following the setup of CaNet [40], we synthetically generate spurious node features to introduce distribution shifts in the Cora and Citeseer datasets. Additionally, from the perspective of graph structure, we select subgraph-level partitioned datasets, as each subgraph has different

Table 2: Node classification performance (%) of VecFormer and three types of GNNs. The best results are in bold, and the second-best results are underlined.

Category	Model	Pubmed	CoraFull	Computer	Photo	CS	Physics
GNN	GCN	87.36 $\pm$ 0.35	67.44 $\pm$ 0.64	91.08 $\pm$ 0.62	94.44 $\pm$ 0.57	93.40 $\pm$ 0.24	96.53 $\pm$ 0.14
	GAT	86.12 $\pm$ 0.44	66.74 $\pm$ 0.75	91.31 $\pm$ 0.45	94.46 $\pm$ 0.69	93.70 $\pm$ 0.29	96.47 $\pm$ 0.19
	SAGE	87.83 $\pm$ 0.34	68.59 $\pm$ 0.80	90.29 $\pm$ 0.68	94.03 $\pm$ 0.69	93.04 $\pm$ 0.26	96.29 $\pm$ 0.17
GT	GOAT	89.15 $\pm$ 0.39	53.30 $\pm$ 3.89	88.95 $\pm$ 1.00	92.11 $\pm$ 0.95	89.96 $\pm$ 0.63	96.27 $\pm$ 0.30
	GraphGPS	87.72 $\pm$ 0.30	59.11 $\pm$ 2.22	87.62 $\pm$ 0.68	94.18 $\pm$ 0.64	<u>95.42 $\pm$ 0.28</u>	<u>97.05 $\pm$ 0.18</u>
	NodeFormer	81.87 $\pm$ 2.24	60.14 $\pm$ 2.97	85.69 $\pm$ 1.25	93.82 $\pm$ 0.97	95.07 $\pm$ 0.39	96.80 $\pm$ 0.63
	SGFormer	90.37 $\pm$ 0.20	<u>70.11 $\pm$ 0.28</u>	90.75 $\pm$ 0.34	94.81 $\pm$ 0.43	95.16 $\pm$ 0.28	96.63 $\pm$ 0.14
	Exphormer	89.06 $\pm$ 0.24	63.70 $\pm$ 0.71	88.85 $\pm$ 0.26	94.10 $\pm$ 0.31	95.03 $\pm$ 0.47	96.93 $\pm$ 0.07
	Polynormer	<u>90.44 $\pm$ 0.21</u>	66.54 $\pm$ 0.89	<u>91.92 $\pm$ 0.25</u>	<u>95.44 $\pm$ 0.24</u>	95.38 $\pm$ 0.32	96.89 $\pm$ 0.06
	NAGphormer	88.65 $\pm$ 0.42	65.51 $\pm$ 0.64	89.71 $\pm$ 0.60	95.03 $\pm$ 0.77	95.23 $\pm$ 0.24	96.86 $\pm$ 0.21
VQ GNN	VQGraph	89.05 $\pm$ 0.32	63.63 $\pm$ 0.84	85.00 $\pm$ 1.04	86.56 $\pm$ 2.39	92.36 $\pm$ 5.87	96.46 $\pm$ 0.20
VQ GT	VecFormer	90.47 $\pm$ 0.28	72.14 $\pm$ 0.67	92.51 $\pm$ 0.54	95.84 $\pm$ 0.41	95.54 $\pm$ 0.34	97.17 $\pm$ 0.24

Table 3: Results of classification accuracy (%) on OOD datasets. The best results are in bold, and the second-best results are underlined. Δ_{OODM} and Δ_{GT} represent the performance improvement of VecFormer compared to the best-performing OOD method and Graph Transformer on each dataset, respectively.

Model	Cora		Citeseer		Twitch			
	OOD	ID	OOD	ID	OOD1	OOD2	OOD3	ID
ERM	91.10 $\pm$ 2.26	95.57 $\pm$ 0.40	82.60 $\pm$ 0.51	89.02 $\pm$ 0.32	65.67 $\pm$ 0.02	52.00 $\pm$ 0.10	61.85 $\pm$ 0.05	75.75 $\pm$ 0.15
IRM	91.63 $\pm$ 1.27	95.72 $\pm$ 0.31	82.73 $\pm$ 0.37	89.11 $\pm$ 0.36	67.27 $\pm$ 0.19	52.85 $\pm$ 0.15	62.40 $\pm$ 0.24	75.30 $\pm$ 0.09
Coral	91.82 $\pm$ 1.30	95.74 $\pm$ 0.39	82.44 $\pm$ 0.58	89.05 $\pm$ 0.37	67.12 $\pm$ 0.03	52.61 $\pm$ 0.01	63.41 $\pm$ 0.01	75.20 $\pm$ 0.01
DANN	92.40 $\pm$ 2.05	95.66 $\pm$ 0.28	82.49 $\pm$ 0.67	89.02 $\pm$ 0.31	66.59 $\pm$ 0.38	52.88 $\pm$ 0.12	62.47 $\pm$ 0.32	75.82 $\pm$ 0.27
GroupDRO	90.54 $\pm$ 0.94	95.38 $\pm$ 0.23	82.64 $\pm$ 0.61	89.13 $\pm$ 0.27	67.41 $\pm$ 0.04	52.99 $\pm$ 0.08	62.29 $\pm$ 0.03	75.74 $\pm$ 0.02
Mixup	92.94 $\pm$ 1.21	94.66 $\pm$ 0.10	82.77 $\pm$ 0.30	89.05 $\pm$ 0.05	65.58 $\pm$ 0.13	52.04 $\pm$ 0.04	61.75 $\pm$ 0.13	75.72 $\pm$ 0.07
SRGNN	91.77 $\pm$ 2.43	95.36 $\pm$ 0.24	82.72 $\pm$ 0.35	89.10 $\pm$ 0.15	66.17 $\pm$ 0.03	52.84 $\pm$ 0.04	62.07 $\pm$ 0.04	75.45 $\pm$ 0.03
EERM	91.80 $\pm$ 0.73	91.37 $\pm$ 0.30	74.07 $\pm$ 0.75	83.53 $\pm$ 0.56	66.80 $\pm$ 0.46	52.39 $\pm$ 0.20	62.07 $\pm$ 0.68	75.19 $\pm$ 0.50
CaNet	<u>97.30 $\pm$ 0.25</u>	95.94 $\pm$ 0.29	<u>95.33 $\pm$ 0.33</u>	89.57 $\pm$ 0.65	<u>68.08 $\pm$ 0.19</u>	53.49 $\pm$ 0.14	<u>63.76 $\pm$ 0.17</u>	<u>76.14 $\pm$ 0.07</u>
NodeFormer	66.81 $\pm$ 1.80	88.48 $\pm$ 3.27	60.00 $\pm$ 1.09	81.15 $\pm$ 0.20	61.75 $\pm$ 0.45	50.68 $\pm$ 0.06	56.99 $\pm$ 0.10	57.80 $\pm$ 0.13
SGFormer	90.18 $\pm$ 0.95	96.29 $\pm$ 0.15	78.26 $\pm$ 0.69	<u>93.95 $\pm$ 0.10</u>	65.11 $\pm$ 0.13	53.63 $\pm$ 0.11	62.20 $\pm$ 0.06	67.90 $\pm$ 0.12
Exphormer	88.76 $\pm$ 0.88	<u>96.35 $\pm$ 0.19</u>	80.54 $\pm$ 1.25	92.14 $\pm$ 0.10	66.22 $\pm$ 0.43	53.69 $\pm$ 0.15	63.26 $\pm$ 0.30	70.54 $\pm$ 0.44
Polynormer	87.00 $\pm$ 1.32	94.96 $\pm$ 0.31	73.95 $\pm$ 1.27	91.95 $\pm$ 0.23	66.07 $\pm$ 0.13	<u>53.90 $\pm$ 0.16</u>	62.93 $\pm$ 0.07	74.48 $\pm$ 0.30
NAGphormer	84.51 $\pm$ 2.29	84.35 $\pm$ 2.35	65.91 $\pm$ 5.21	84.86 $\pm$ 1.20	62.85 $\pm$ 2.88	52.10 $\pm$ 1.41	59.01 $\pm$ 3.37	71.42 $\pm$ 1.61
VecFormer	98.85 $\pm$ 0.08	97.91 $\pm$ 0.14	97.45 $\pm$ 0.04	94.94 $\pm$ 0.20	68.14 $\pm$ 0.14	54.08 $\pm$ 0.16	64.27 $\pm$ 0.18	76.51 $\pm$ 0.09
Δ_{OODM}	+1.55%	+1.97%	+2.12%	+5.37%	+0.06%	+0.59%	+0.51%	+0.37%
Δ_{GT}	+8.67%	+1.56%	+16.91%	+0.99%	+1.92%	+0.18%	+1.01%	+2.03%

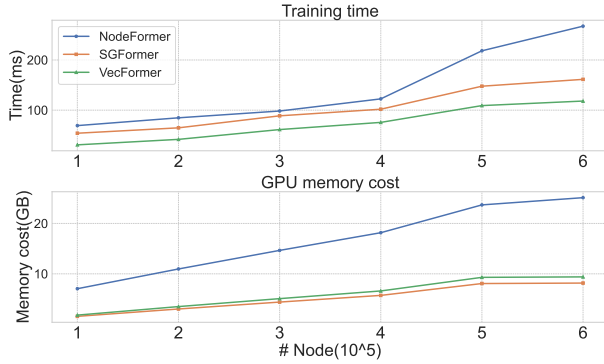
sizes, densities, and node degrees. For each dataset, we divide the ID dataset into training, validation, and test sets with ratios of 50%, 25%, and 25%, respectively. The trained model is then tested on both ID test set and the OOD set. Each experiment is run five times, and we report the accuracy on the Cora and Citeseer OOD datasets and the ROC AUC on the Twitch dataset.

Baselines. We select two types of methods as baselines for comparison: GT and graph OOD methods such as ERM [35], IRM [1], Coral [33], DANN [13], GroupDRO [28], Mixup [55], SR-GNN [58], EERM [41], and CaNet [40] (**the SOTA Graph OOD method**). We use the GAT as the encoder backbone.

Results. From Table 3, we can observe that: (1) Graph Transformers perform better on ID data but underperform on OOD data compared to Graph OOD generalization methods, due to overfitting on ID data. (2) Graph Transformers perform worse on node distribution shift OOD datasets, as the significant difference between OOD and ID node features validates the inference in Section 4.3. (3) On all ID and OOD test sets VecFormer outperforms the SOTA Graph OOD generalization method, CaNet. Moreover, compared to the best-performing Graph Transformer, VecFormer achieves improvements of **8.67%** and **16.91%** on the Cora OOD and Citeseer

Table 4: Performance comparison (%) of different methods on large datasets. The best results are in bold, and the second-best results are underlined.

Model	Ogbn-proteins	Amazon2m	Pokec
# nodes	132,534	2,449,029	1,632,803
# edges	39,561,252	61,859,140	30,622,564
MLP	72.04 $\pm$ 0.48	63.46 $\pm$ 0.10	60.15 $\pm$ 0.03
GCN	72.51 $\pm$ 0.35	83.90 $\pm$ 0.10	62.31 $\pm$ 1.13
SGC	70.31 $\pm$ 0.23	81.21 $\pm$ 0.12	52.03 $\pm$ 0.84
GCN-NSampler	73.51 $\pm$ 1.31	83.84 $\pm$ 0.42	63.75 $\pm$ 0.77
GAT-NSampler	74.63 $\pm$ 1.24	85.17 $\pm$ 0.32	62.32 $\pm$ 0.65
SIGN	71.24 $\pm$ 0.46	80.98 $\pm$ 0.31	68.01 $\pm$ 0.25
NodeFormer	77.45 $\pm$ 1.15	87.85 $\pm$ 0.24	70.32 $\pm$ 0.45
SGFormer	79.53 $\pm$ 0.38	89.09 $\pm$ 0.10	73.76 $\pm$ 0.24
VecFormer	80.52 $\pm$ 0.20	89.72 $\pm$ 0.04	78.06 $\pm$ 0.21

**Figure 4: The training time per epoch and GPU memory usage of three linear-complexity graph transformers on graphs with varying numbers of nodes.**

OOD test sets, demonstrating the generalization capability of graph token attention.

5.3 Performance on Large-sized Graph (RQ3)

Setup. We evaluate VecFormer from three aspects: model performance, training time, and GPU memory consumption. We assess VecFormer’s performance on large graphs with sizes ranging from 0.1M to 2M, including Ogbn-proteins [17], Pokec [22], and Amazon2m [6]. The dataset partitioning follows the settings in the SGFormer [43]. To verify the scalability of the model, we sample subgraphs with node sizes ranging from 100K to 600K from Amazon2m as datasets, and record the training time and GPU memory consumption per epoch.

Baselines. We select standard GNNs, GCN and GAT, sampling-based GNNs [54], GCN-NSampler and GAT-NSampler, scalable GNNs, SGC [37] and SIGN [11], and two representative linear-complexity Graph Transformers, NodeFormer [42] and SGFormer [43], as baselines.

Results. From Figure 4, we observe that the training time and GPU memory consumption of the three linear-time-complexity

Graph Transformers differ. VecFormer and SGFormer have significantly lower training time and memory consumption compared to NodeFormer. VecFormer has a shorter training time than SGFormer, with slightly higher memory consumption. In terms of model performance, as shown in Table 4, VecFormer significantly outperforms the other two Graph Transformers on all three datasets. From the three evaluation perspectives, VecFormer demonstrates superior model performance and training time compared to NodeFormer and SGFormer, proving the scalability of graph token attention.

5.4 Effectiveness Analysis (RQ4)

In this section, we analyze the effectiveness of VecFormer through Graph Codes and Graph Tokens.

Graph Codes. As the fundamental unit of vector quantization, Graph Codes influence the Graph Codebook Training phase. We set the sizes of the feature codebook and structure codebook to 2, 4, 8, 16, and 32, and record the values of various reconstruction losses during training. From Figure 5, we observe that as the number of codes in the codebooks increases, reconstruction losses gradually decrease until stabilize.

Graph Tokens. The Graph Token List serves to compress global graph information, enabling nodes to interact with it through cross-attention mechanisms to capture global context information. As shown in Figure 6, when the size of the Graph Token List is set to 4, 16, 64, or 256, the model performs poorly with a smaller number of Graph Tokens, as it struggles to effectively capture and compress graph information. As the number of Graph Tokens increases, the model’s performance gradually improves and stabilizes. In the meantime, to verify whether VecFormer learns beneficial attention patterns, we visualize dimensionality-reduced attention weights (rather than model outputs) between nodes and Graph Token List on OOD datasets. As can be observed from Figure 7, the attention weights correlate with labels and exhibit non-uniform distributions, proving that VecFormer learns meaningful contextual patterns.

6 Conclusion

In this work, we focus on improving the GT’s computational efficiency and performance in OOD scenarios. We introduce a variant of VQ, SoftVQ, which is applied to GT to compress the distribution on graphs to reduce computational complexity. In the meantime, both theoretical and empirical evidence demonstrate the superiority of VecFormer in OOD scenarios. We believe this paper will open new directions for designing more efficient and generalizable Graph Transformers. Interesting directions for future work include: (1) exploring GT in graph imbalance scenarios; and (2) designing graph foundation models based on vector quantization.

Acknowledgments

This work was supported by National Science and Technology Innovation - Major Program(No. 2022ZD0115101), the National Science and Technology Major Project of China (No.2021YFA1301603), and the Project (No. WU2025B006) from the SOE Dean Special Project Fund (SOE-DSPF) Program of Westlake University.

References

- [1] Martin Arjovsky, Léon Bottou, Ishaan Gulrajani, and David Lopez-Paz. 2020. Invariant Risk Minimization. *arXiv:1907.02893 [stat.ML]* <https://arxiv.org/abs/1907.02893>
- [2] Jinsong Chen, Kaiyuan Gao, Gaichao Li, and Kun He. 2023. NAGphormer: A Tokenized Graph Transformer for Node Classification in Large Graphs. *arXiv:2206.04910 [cs.LG]* <https://arxiv.org/abs/2206.04910>
- [3] Jiabei Cheng, Changxi Chi, Jingbo Zhou, Hongyi Xin, and Jun Xia. [n. d.]. PRE-SCRIBE: Predicting Single-Cell Responses with Bayesian Estimation. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- [4] Jiabei Cheng, Jiachen Li, Kaiyuan Yang, Hongbin Shen, and Ye Yuan. 2024. GenoHopton: Bridging Gene Network Graphs and Single-Cell Foundation Models. *arXiv:2411.06331 [q-bio.QM]* <https://arxiv.org/abs/2411.06331>
- [5] Changxi Chi, Jun Xia, Jingbo Zhou, Jiabei Cheng, Chang Yu, and Stan Z Li. 2025. GRAPE: Heterogeneous Graph Representation Learning for Genetic Perturbation with Coding and Non-Coding Biotype. *arXiv preprint arXiv:2505.03853* (2025).
- [6] Wei-Lin Chiang, Xuanqing Liu, Si Si, Yang Li, Samy Bengio, and Cho-Jui Hsieh. 2019. Cluster-GCN: An Efficient Algorithm for Training Deep and Large Graph Convolutional Networks. In *the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '19)*. ACM. doi:10.1145/3292500.3330925
- [7] Jiang dejun, Zhenxing Wu, Kim Hsieh, Chen Guangyong, Ben Liao, Zhe Wang, Chao Shen, Dong-Sheng Cao, Jian Wu, and Tingjun Hou. 2020. Could Graph Neural Networks Learn Better Molecular Representation for Drug Discovery? A Comparison Study of Descriptor-based and Graph-based Models. (09 2020). doi:10.21203/rs.3.rs-79416/v1
- [8] Chenhui Deng, Zichao Yue, and Zhiru Zhang. 2024. Polynormer: Polynomial-Expressive Graph Transformer in Linear Time. *arXiv:2403.01232 [cs.LG]* <https://arxiv.org/abs/2403.01232>
- [9] Mucong Ding, Kezhi Kong, Jingling Li, Chen Zhu, John P Dickerson, Furong Huang, and Tom Goldstein. 2021. VQ-GNN: A Universal Framework to Scale up Graph Neural Networks using Vector Quantization. *arXiv:2110.14363 [cs.LG]* <https://arxiv.org/abs/2110.14363>
- [10] Vijay Prakash Dwivedi, Ladislav Rampásek, Mikhail Galkin, Ali Parviz, Guy Wolf, Anh Tuan Luu, and Dominique Beaini. 2023. Long Range Graph Benchmark. *arXiv:2206.08164 [cs.LG]* <https://arxiv.org/abs/2206.08164>
- [11] Fabrizio Frasca, Emanuele Rossi, Davide Eynard, Ben Chamberlain, Michael Bronstein, and Federico Monti. 2020. SIGN: Scalable Inception Graph Neural Networks. *arXiv:2004.11198 [cs.LG]* <https://arxiv.org/abs/2004.11198>
- [12] Dongqi Fu, Zhigang Hua, Yan Xie, Jin Fang, Si Zhang, Kaan Sancak, Hao Wu, Andrey Malevich, Jingrui He, and Bo Long. 2024. VCR-Graphormer: A Mini-batch Graph Transformer via Virtual Connections. *arXiv:2403.16030 [cs.LG]* <https://arxiv.org/abs/2403.16030>
- [13] Yaroslav Ganin, Evgeniya Ustinova, Hana Ajakan, Pascal Germain, Hugo Larochelle, François Laviolette, Mario Marchand, and Victor Lempitsky. 2016. Domain-Adversarial Training of Neural Networks. *arXiv:1505.07818 [stat.ML]* <https://arxiv.org/abs/1505.07818>
- [14] Zhangyang Gao, Cheng Tan, Jue Wang, Yufei Huang, Lirong Wu, and Stan Z. Li. 2024. FoldToken: Learning Protein Language via Vector Quantization and Beyond. *arXiv:2403.09673 [q-bio.BM]* <https://arxiv.org/abs/2403.09673>
- [15] William L. Hamilton, Rex Ying, and Jure Leskovec. 2018. Inductive Representation Learning on Large Graphs. *arXiv:1706.02216 [cs.SI]* <https://arxiv.org/abs/1706.02216>
- [16] Zhenyu Hou, Xiao Liu, Yukuo Cen, Yuxiao Dong, Hongxia Yang, Chunjie Wang, and Jie Tang. 2022. GraphMAE: Self-Supervised Masked Graph Autoencoders. *arXiv:2205.10803 [cs.LG]* <https://arxiv.org/abs/2205.10803>
- [17] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2021. Open Graph Benchmark: Datasets for Machine Learning on Graphs. *arXiv:2005.00687 [cs.LG]* <https://arxiv.org/abs/2005.00687>
- [18] Jiawei Jiang, Jiacheng Chen, Honghui Xu, Yuhang He, and Jianwei Zheng. 2025. Information-coupled MRI acceleration via multi-modal mapping and progressive masking. *Pattern Recognition* (2025), 112061.
- [19] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. *arXiv:1609.02907 [cs.LG]* <https://arxiv.org/abs/1609.02907>
- [20] Kezhi Kong, Jiuhai Chen, John Kirchenbauer, Renkun Ni, C. Bayan Bruss, and Tom Goldstein. 2023. GOAT: A Global Transformer on Large-scale Graphs. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*. Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (Eds.). PMLR, 17375–17390. <https://proceedings.mlr.press/v202/kong23a.html>
- [21] Kezhi Kong, Jiuhai Chen, John Kirchenbauer, Renkun Ni, C. Bayan Bruss, and Tom Goldstein. 2023. GOAT: A Global Transformer on Large-scale Graphs.
- [22] Jure Leskovec and Andrej Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection. (Jun 2014).
- [23] Andrzej Maćkiewicz and Waldemar Ratajczak. 1993. Principal components analysis (PCA). *Computers & Geosciences* 19, 3 (1993), 303–342.
- [24] Hongbin Pei, Bingzhe Wei, Kevin Chen-Chuan Chang, Yu Lei, and Bo Yang. 2020. Geom-GCN: Geometric Graph Convolutional Networks. *arXiv:2002.05287 [cs.LG]* <https://arxiv.org/abs/2002.05287>
- [25] Oleg Platonov, Denis Kuznedelev, Michael Diskin, Artem Babenko, and Liudmila Prokhorenkova. 2024. A critical look at the evaluation of GNNs under heterophily: Are we really making progress? *arXiv:2302.11640 [cs.LG]* <https://arxiv.org/abs/2302.11640>
- [26] Ladislav Rampásek, Mikhail Galkin, Vijay Prakash Dwivedi, Anh Tuan Luu, Guy Wolf, and Dominique Beaini. 2023. Recipe for a General, Powerful, Scalable Graph Transformer. *arXiv:2205.12454 [cs.LG]* <https://arxiv.org/abs/2205.12454>
- [27] Yusuf H Roohani, Tony J Hua, Po-Yuan Tung, Lexi R Bounds, Feiqiao B Yu, Alexander Dobin, Noam Teyssier, Abhinav Adduri, Alden Woodrow, Brian S Plosky, et al. 2025. Virtual Cell Challenge: Toward a Turing test for the virtual cell. *Cell* 188, 13 (2025), 3370–3374.
- [28] Shiori Sagawa, Pang Wei Koh, Tatsunori B. Hashimoto, and Percy Liang. 2020. Distributionally Robust Neural Networks for Group Shifts: On the Importance of Regularization for Worst-Case Generalization. *arXiv:1911.08731 [cs.LG]* <https://arxiv.org/abs/1911.08731>
- [29] Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Gallagher, and Tina Eliassi-Rad. 2008. Collective Classification in Network Data. 93–106 pages. doi:10.1609/aimag.v29i3.2157
- [30] Oleksandr Shchur, Maximilian Mumme, Aleksandar Bojchevski, and Stephan Günnemann. 2019. Pitfalls of Graph Neural Network Evaluation. *arXiv:1811.05868 [cs.LG]* <https://arxiv.org/abs/1811.05868>
- [31] Hamed Shirzad, Ameya Velingker, Balaji Venkatachalam, Danica J. Sutherland, and Ali Kemal Sinop. 2023. Exphormer: Sparse Transformers for Graphs. *arXiv:2303.06147 [cs.LG]* <https://arxiv.org/abs/2303.06147>
- [32] Alexey Strokach, David Becerra, Carles Corbi-Verge, Albert Perez-Riba, and Philip M. Kim. 2020. Fast and Flexible Protein Design Using Deep Graph Neural Networks. *Cell systems* (2020). <https://api.semanticscholar.org/CorpusID:221912492>
- [33] Baochen Sun and Kate Saenko. 2016. Deep CORAL: Correlation Alignment for Deep Domain Adaptation. *arXiv:1607.01719 [cs.CV]* <https://arxiv.org/abs/1607.01719>
- [34] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. 2018. Neural Discrete Representation Learning. *arXiv:1711.00937 [cs.LG]* <https://arxiv.org/abs/1711.00937>
- [35] V.N. Vapnik. 1999. An overview of statistical learning theory. *IEEE Transactions on Neural Networks* 10, 5 (1999), 988–999. doi:10.1109/72.788640
- [36] Petar Velićković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. *arXiv:1710.10903 [stat.ML]* <https://arxiv.org/abs/1710.10903>
- [37] Felix Wu, Tianyi Zhang, Amauri Holanda de Souza Jr., Christopher Fifty, Tao Yu, and Kilian Q. Weinberger. 2019. Simplifying Graph Convolutional Networks. *arXiv:1902.07153 [cs.LG]* <https://arxiv.org/abs/1902.07153>
- [38] Lirong Wu, Yijun Tian, Yufei Huang, Siyuan Li, Haitao Lin, Nitesh V Chawla, and Stan Z. Li. 2024. MAPE-PPI: Towards Effective and Efficient Protein-Protein Interaction Prediction via Microenvironment-Aware Protein Embedding. *arXiv:2402.14391 [cs.LG]* <https://arxiv.org/abs/2402.14391>
- [39] Lirong Wu, Yijun Tian, Haitao Lin, Yufei Huang, Siyuan Li, Nitesh V Chawla, and Stan Z. Li. 2024. Learning to Predict Mutation Effects of Protein-Protein Interactions by Microenvironment-aware Hierarchical Prompt Learning. *arXiv:2405.10348 [q-bio.QM]* <https://arxiv.org/abs/2405.10348>
- [40] Qitian Wu, Fan Nie, Chenxiao Yang, Tianyi Bao, and Junchi Yan. 2024. Graph Out-of-Distribution Generalization via Causal Intervention. *arXiv:2402.11494 [cs.LG]* <https://arxiv.org/abs/2402.11494>
- [41] Qitian Wu, Hengrui Zhang, Junchi Yan, and David Wipf. 2024. Handling Distribution Shifts on Graphs: An Invariance Perspective. *arXiv:2202.02466 [cs.LG]* <https://arxiv.org/abs/2202.02466>
- [42] Qitian Wu, Wentao Zhao, Zenan Li, David Wipf, and Junchi Yan. 2023. NodeFormer: A Scalable Graph Structure Learning Transformer for Node Classification. *arXiv:2306.08385 [cs.LG]* <https://arxiv.org/abs/2306.08385>
- [43] Qitian Wu, Wentao Zhao, Chenxiao Yang, Hengrui Zhang, Fan Nie, Haitian Jiang, Yatao Bian, and Junchi Yan. 2024. SGFormer: Simplifying and Empowering Transformers for Large-Graph Representations. *arXiv:2306.10759 [cs.LG]* <https://arxiv.org/abs/2306.10759>
- [44] Zhanghao Wu, Paras Jain, Matthew A. Wright, Azalia Mirhoseini, Joseph E. Gonzalez, and Ion Stoica. 2022. Representing Long-Range Context for Graph Neural Networks with Global Attention. *arXiv:2201.08821 [cs.LG]* <https://arxiv.org/abs/2201.08821>
- [45] Jun Xia, Chengshuai Zhao, Bozhen Hu, Zhangyang Gao, Cheng Tan, Yue Liu, Siyuan Li, and Stan Z. Li. 2023. Mole-BERT: Rethinking Pre-training Graph Neural Networks for Molecules. In *International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=jevY-DtiZTR>
- [46] Yujie Xing, Xiao Wang, Yibo Li, Hai Huang, and Chuan Shi. 2024. Less is More: on the Over-Globalizing Problem in Graph Transformers. *arXiv:2405.01102 [cs.LG]*

- <https://arxiv.org/abs/2405.01102>
- [47] Yan Xu, Yu Lu, Changtao Ji, and Qiyuan Zhang. 2023. Adaptive Graph Fusion Convolutional Recurrent Network for Traffic Forecasting. *IEEE Internet Things J.* 10, 13 (2023), 11465–11475. doi:10.1109/IJOT.2023.3244182
 - [48] Ling Yang, Ye Tian, Minkai Xu, Zhongyi Liu, Shenda Hong, Wei Qu, Wentao Zhang, Bin Cui, Muhan Zhang, and Jure Leskovec. 2024. VQGraph: Rethinking Graph Representation Space for Bridging GNNs and MLPs. arXiv:2308.02117 [cs.LG] <https://arxiv.org/abs/2308.02117>
 - [49] Shenzhi Yang, Bin Liang, An Liu, Lin Gui, Xingkai Yao, and Xiaofang Zhang. 2025. Bounded and uniform energy-based out-of-distribution detection for graphs. *arXiv preprint arXiv:2504.13429* (2025).
 - [50] Shenzhi Yang, Jun Xia, Jingbo Zhou, Xingkai Yao, and Xiaofang Zhang. 2025. NodeReg: Mitigating the Imbalance and Distribution Shift Effects in Semi-Supervised Node Classification via Norm Consistency. *arXiv preprint arXiv:2503.03211* (2025).
 - [51] Shenzhi Yang, Junbo Zhao, Sharon Li, Shouqing Yang, Dingyu Yang, Xiaofang Zhang, and Haobo Wang. [n. d.]. Harnessing Feature Resonance under Arbitrary Target Alignment for Out-of-Distribution Node Detection. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
 - [52] Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, and Tie-Yan Liu. 2021. Do Transformers Really Perform Bad for Graph Representation? arXiv:2106.05234 [cs.LG] <https://arxiv.org/abs/2106.05234>
 - [53] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L. Hamilton, and Jure Leskovec. 2018. Graph Convolutional Neural Networks for Web-Scale Recommender Systems. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD 2018, London, UK, August 19-23, 2018*, Yike Guo and Faisal Farooq (Eds.). ACM, 974–983. doi:10.1145/3219819.3219890
 - [54] Hanqing Zeng, Hongkuan Zhou, Ajitesh Srivastava, Rajgopal Kannan, and Viktor Prasanna. 2020. GraphSAINT: Graph Sampling Based Inductive Learning Method. arXiv:1907.04931 [cs.LG] <https://arxiv.org/abs/1907.04931>
 - [55] Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, and David Lopez-Paz. 2018. mixup: Beyond Empirical Risk Minimization. arXiv:1710.09412 [cs.LG] <https://arxiv.org/abs/1710.09412>
 - [56] Jiawei Zhang, Haopeng Zhang, Congying Xia, and Li Sun. 2020. GraphBert: Only Attention is Needed for Learning Graph Representations. arXiv:2001.05140 [cs.LG] <https://arxiv.org/abs/2001.05140>
 - [57] Jingbo Zhou, Yixuan Du, Ruqiong Zhang, Jun Xia, Zhizhi Yu, Zelin Zang, Di Jin, Carl Yang, Rui Zhang, and Stan Z Li. 2024. Deep Graph Neural Networks via Posterior-Sampling-based Node-Adaptive Residual Module. *Advances in Neural Information Processing Systems* 37 (2024), 68211–68238.
 - [58] Qi Zhu, Natalia Ponomareva, Jiawei Han, and Bryan Perozzi. 2021. Shift-Robust GNNs: Overcoming the Limitations of Localized Graph Training Data. arXiv:2108.01099 [cs.LG] <https://arxiv.org/abs/2108.01099>
 - [59] Wenhao Zhu, Guojie Song, Liang Wang, and Shaoguo Liu. 2024. AnchorGT: Efficient and Flexible Attention Architecture for Scalable Graph Transformers. arXiv:2405.03481 [cs.LG] <https://arxiv.org/abs/2405.03481>

A Design Details & Hyper-parameter Settings

In this section, we introduce the design details of our proposed method and summarize the hyper-parameter settings. In VecFormer, a GAT with residual connections is employed as the GNN encoder, while the number of cross-attention layers is set to 1. The Feature Decoder, Structure Decoder, and Graph Decoder are all single-layer MLPs. The training hyperparameters of VecFormer are selected using grid search with the following search space: learning rate $\{0.001, 0.005, 0.01\}$, hidden size $\{64, 128, 256\}$, weight decay $\{1e-3, 5e-4, 1e-4\}$, and dropout ratio $\{0.1, 0.3, 0.5, 0.7\}$. The hyperparameters related to the number of codes and tokens are shown in Table 5.

Table 5: Hyper-parameter settings of our proposed method.

Dataset	Feature Codebook Size	Structure Codebook Size	N_f	N_s
Pubmed	256	256	64	64
CoraFull	256	256	64	64
Computer	256	256	64	64
Photo	256	256	64	64
CS	256	256	64	64
Physics	256	256	64	64
Chameleon	256	256	32	32
Squirrel	128	128	32	32
Cora OOD	64	64	16	16
Citeseer OOD	64	64	16	16
Twitch	512	512	64	64
Ogbn-proteins	64	64	16	16
Amazon2m	256	256	64	64
Pokec	256	256	64	64

B Datasets

Table 6: The statistical information of eight datasets.

Dataset	# Nodes	# Edges	# Feature Dims	# Classes
Pubmed	19,717	44,324	500	3
CoraFull	19,793	126,842	8,710	70
Computer	13,752	245,861	767	10
Photo	7,650	119,081	745	8
CS	18,333	81,894	6,805	15
Physics	34,493	247,962	8,415	5
Chameleon	890	8,854	2,325	5
Squirrel	2,223	46,998	2,089	5

C Application in Gene Perturbation Prediction

In recent years, graph neural networks have demonstrated strong capability in modeling complex relational structures, making them particularly suitable for biological network scenarios where high-dimensional dependencies can be represented as nodes and edges. Regulatory and co-expression relationships among genes naturally

form graph structures, and gene perturbations can trigger cascading effects over the network, ultimately manifested as a set of differentially expressed (DE) genes. Therefore, we formulate gene-perturbation response prediction as a node classification task on a gene co-expression network: given a perturbation condition, we predict for each gene whether it is differentially expressed, thereby leveraging GNNs’ strengths in graph information propagation and representation learning to better model perturbation effects.

Dataset and Task. We evaluate our method on the Virtual Cell Challenge (VCC) [27] training dataset, which contains single-cell RNA-seq data from 221,273 cells and 18,080 genes. The dataset includes 150 gene-perturbation (gene knockout) experiments and 38,176 non-targeting control cells. We model the problem as a node classification task on a gene co-expression network, aiming to predict DE genes for each perturbation experiment.

Data Split. Using a fixed random seed, we randomly split the 150 perturbations into a training set (80), a validation set (20), and a test set (50). The validation set is used for model selection (checkpoint selection), and the test set is used only for final evaluation.

Graph Construction. We construct a symmetric k -nearest-neighbor (k -NN) co-expression network based on Pearson correlation coefficients computed from the control expression matrix. We experiment with $k \in \{5, 10, 20\}$ to study the effect of graph density on model performance.

Node Features. For each perturbation experiment, node features are constructed as follows: (1) we apply PCA [23] to the control gene expression matrix (gene $\times$ cell) to reduce the dimension to 128, obtaining a base representation for each gene; (2) for a specific perturbation targeting gene g , we add g ’s PCA embedding to the representations of all genes, enabling the model to perceive which gene is perturbed.

Label Generation. We perform the Wilcoxon rank-sum test to compare each gene’s expression between perturbed cells and control cells, followed by Benjamini–Hochberg FDR correction. Genes with adjusted p -values < 0.05 are labeled as differentially expressed. We consider a binary classification setting.

Evaluation Metric. We report the Differential Expression Score (DES): following the official VCC metric, DES measures the overlap between the predicted DE gene set and the ground-truth DE gene set, normalized by the number of true DE genes. When the predicted set is larger than the true set, we select the top- k genes with the highest predicted DE probabilities.

Baselines and Implementation. We compare four models: GCN, GAT, and VecFormer (our method). All models use 2 layers with hidden dimension 128 and dropout rate 0.2. VecFormer is trained in two stages: a pretraining stage with a reconstruction loss for 100 epochs, and a classification stage trained for up to 300 epochs with early stopping based on validation performance.

Results. Table 7 reports the Differential Expression Score (DES) under different k -NN graph densities. Overall, VecFormer consistently improves over its corresponding GNN backbone across all graph settings, demonstrating the effectiveness of the proposed perturbation-aware representation learning.

For the GCN backbone, VecFormer achieves higher mean DES for all k . The gain is modest on the sparse graph ($k = 5$), improving from 0.1286 to 0.1490 ($\Delta\text{DES} = +0.0204$, $+15.9\%$). As the graph becomes denser, the improvement becomes substantially larger: at

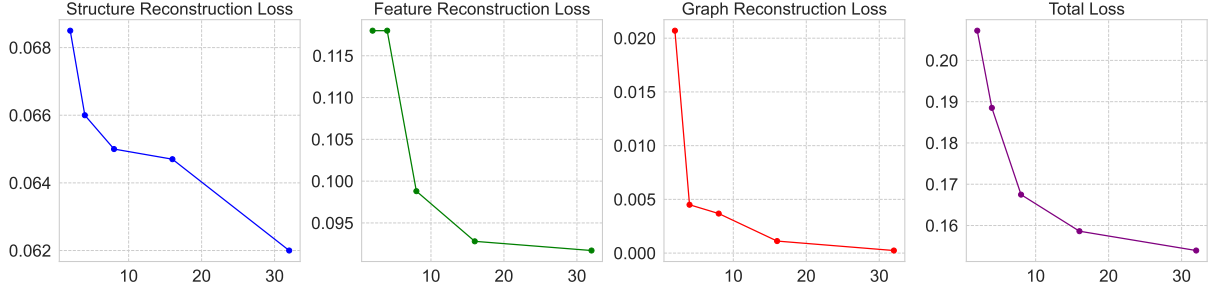


Figure 5: Reconstruction losses of different types and the total loss under varying numbers of Graph Codes.

Table 7: DES (mean $\pm$ std) on the VCC test set under different k -NN graph densities. We additionally report the mean improvement of VecFormer over its corresponding backbone: Δ DES (absolute gain in mean DES) with relative improvement in parentheses.

Model	$k = 5$	$k = 10$	$k = 20$
GCN	0.1286 $\pm$ 0.1462	0.0085 $\pm$ 0.0420	0.0031 $\pm$ 0.0148
VecFormer (GCN backbone)	0.1490 $\pm$ 0.2251	0.0699 $\pm$ 0.1566	0.1000 $\pm$ 0.1989
Δ DES vs. GCN	+0.0204 (+15.9%)	+0.0614 (+722.4%)	+0.0969 (+3125.8%)
GAT	0.0316 $\pm$ 0.0197	0.0244 $\pm$ 0.0148	0.0445 $\pm$ 0.0358
VecFormer (GAT backbone)	0.1322 $\pm$ 0.2226	0.1298 $\pm$ 0.2045	0.1445 $\pm$ 0.1829
Δ DES vs. GAT	+0.1006 (+318.4%)	+0.1054 (+432.8%)	+0.1000 (+224.7%)

$k = 10$, VecFormer increases DES from 0.0085 to 0.0699 (+0.0614, +722.4%), and at $k = 20$ from 0.0031 to 0.1000 (+0.0969, +3125.8%). These results suggest that GCN is sensitive to graph density, while VecFormer benefits from richer neighborhood structure.

A similar trend is observed with the GAT backbone. VecFormer yields consistent improvements over GAT across all densities: from 0.0316 to 0.1322 at $k = 5$ (+0.1006, +318.4%), from 0.0244 to 0.1298 at $k = 10$ (+0.1054, +432.8%), and from 0.0445 to 0.1445 at $k = 20$ (+0.1000, +224.7%). Notably, VecFormer with a GAT backbone attains the best performance, peaking at 0.1445 DES when $k = 20$.

In summary, VecFormer provides substantial accuracy gains on DE gene prediction and exhibits strong robustness to graph construction choices, indicating improved capability in capturing perturbation-induced effects on gene co-expression networks.

D Effectiveness Analysis

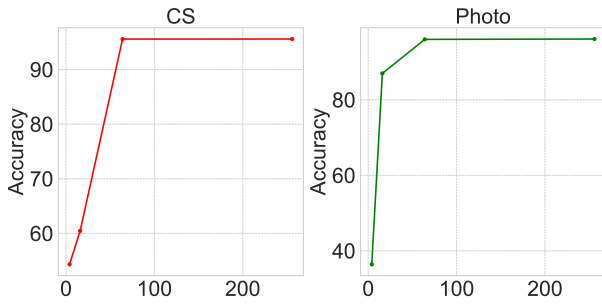


Figure 6: VecFormer's performance(%) under varying numbers of Graph Tokens.

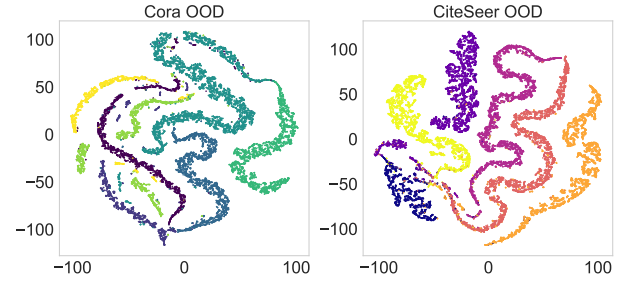


Figure 7: t -SNE visualization of attention weights on OOD datasets.